



Tutorial Lengkap Pengembangan Sistem TERDEPAN

BAPPEDA Kuningan - Complete Guide Modul 1-7

Sistem: Terintegrasi, Responsif, Dinamis, Efektif, Profesional, Akuntabel, iNovatif

Framework: Laravel 11 + nwidart/laravel-modules

Database: MySQL 8.0+

Total Modul: 7 Modul

Total Tabel: 32 Tabel



DAFTAR ISI

1. Persiapan Awal
2. Modul 1: Inti
3. Modul 2: Dokumen
4. Modul 3: Perjanjian Kinerja
5. Modul 4: Penugasan
6. Modul 5: Evaluasi
7. Modul 6: Sistem
8. Modul 7: Dashboard
9. Testing & Deployment



PERSIAPAN AWAL

1. Install Laravel & Dependencies

```
bash

# Install Laravel 11
composer create-project laravel/laravel terdepan "11.*"

cd terdepan

# Install Laravel Modules
composer require nwidart/laravel-modules

# Publish config
php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider"

# Install dependencies tambahan
composer require barryvdh/laravel-dompdf
composer require maatwebsite/excel
composer require intervention/image
composer require spatie/laravel-permission
composer require laravel/sanctum
```

2. Setup Database

File: `.env`

```
env

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=terdepan_bappeda
DB_USERNAME=root
DB_PASSWORD=

APP_NAME="TERDEPAN BAPPEDA"
APP_URL=http://localhost:8000
APP_TIMEZONE=Asia/Jakarta
```

```
bash

# Buat database
mysql -u root -p
CREATE DATABASE terdepan_bappeda CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
EXIT;

# Test koneksi
php artisan migrate:status
```

3. Setup Folder Structure

```
bash

# Buat folder storage
mkdir -p storage/app/public/dokumen/{surat_masuk,surat_keluar,laporan,spasial}
mkdir -p storage/app/public/ttd_digital
mkdir -p storage/app/public/foto_bukti
mkdir -p storage/app/public/perjanjian_kinerja
mkdir -p storage/app/public/foto_profile

# Symlink storage
php artisan storage:link

# Set permissions
chmod -R 775 storage bootstrap/cache
```

4. Git Setup

```
bash

git init
git add .
git commit -m "Initial commit - Laravel setup"
git checkout -b development
```

 MODUL 1: INTI

Priority: CRITICAL ★★★★★

Estimasi: 3-5 hari

Tabel: 4 (master_jabatan, master_bidang, master_pegawai, master_ttd_digital)

Generate Modul

```
bash

php artisan module:make Inti
```

Buat Migration

```
bash

php artisan module:make-migration create_master_jabatan_table Inti
php artisan module:make-migration create_master_bidang_table Inti
php artisan module:make-migration create_master_pegawai_table Inti
php artisan module:make-migration create_master_ttd_digital_table Inti
```

Migration Files

create_master_jabatan_table.php

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('master_jabatan', function (Blueprint $table) {
            $table->id();
            $table->string('kode', 10)->unique();
            $table->string('nama', 100);
            $table->tinyInteger('level');
            $table->boolean('is_struktural')->default(true);
            $table->boolean('bebas_nilai_kinerja')->default(false);
            $table->boolean('is_active')->default(true);
            $table->timestamps();

            $table->index(['kode', 'is_active']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('master_jabatan');
    }
};
```

create_master_bidang_table.php

```
php
```

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('master_bidang', function (Blueprint $table) {
            $table->id();
            $table->string('kode', 20)->unique();
            $table->string('nama', 100);
            $table->text('deskripsi')->nullable();
            $table->string('warna', 7)->default('#2563eb');
            $table->boolean('is_active')->default(true);
            $table->timestamps();
            $table->softDeletes();

            $table->index(['kode', 'is_active']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('master_bidang');
    }
};
```

create_master_pegawai_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('master_pegawai', function (Blueprint $table) {
            $table->id();
            $table->string('nomor_identitas', 18)->unique();
            $table->enum('tipe_identitas', ['NIP', 'NIK']->default('NIP');
            $table->string('nama', 100);
            $table->foreignId('jabatan_id')->constrained('master_jabatan')->cascadeOnUpdate()->restrictOnDelete();
            $table->foreignId('bidang_id')->constrained('master_bidang')->cascadeOnUpdate()->restrictOnDelete();
            $table->string('email')->unique();
            $table->string('password');
            $table->string('no_telepon', 15)->nullable();
            $table->string('pangkat', 50)->nullable();
            $table->string('golongan', 10)->nullable();
            $table->string('gelar_depan', 20)->nullable();
            $table->string('gelar_belakang', 50)->nullable();
            $table->date('tanggal_lahir')->nullable();
            $table->enum('jenis_kelamin', ['L', 'P']);
            $table->text('alamat')->nullable();
            $table->enum('status_kepegawaian', ['PNS', 'PPPK', 'Kontrak']->default('PNS');
            $table->enum('status_aktif', ['Aktif', 'Nonaktif', 'Cuti', 'Pensiun']->default('Aktif');
            $table->date('tanggal_masuk')->nullable();
            $table->date('tanggal_keluar')->nullable();
            $table->foreignId('atasan_langsung_id')->nullable()->constrained('master_pegawai')->nullOnDelete();
            $table->string('foto_profile_path')->nullable();
            $table->timestamp('last_login_at')->nullable();
            $table->string('last_login_ip', 45)->nullable();
            $table->rememberToken();
            $table->timestamps();
            $table->softDeletes();

            $table->index(['nomor_identitas', 'email', 'status_aktif']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('master_pegawai');
    }
};
```

create_master_ttd_digital_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('master_ttd_digital', function (Blueprint $table) {
            $table->id();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->cascadeOnDelete();
            $table->string('file_path');
            $table->string('file_hash', 64);
            $table->integer('image_width');
            $table->integer('image_height');
            $table->date('valid_from');
            $table->date('valid_until')->nullable();
            $table->boolean('is_active')->default(true);
            $table->boolean('is_verified')->default(false);
            $table->timestamps();
            $table->softDeletes();

            $table->index(['pegawai_id', 'is_active']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('master_ttd_digital');
    }
};
```

Buat Models

```
bash

php artisan module:make-model Jabatan Inti
php artisan module:make-model Bidang Inti
php artisan module:make-model Pegawai Inti
php artisan module:make-model TtdDigital Inti
```

Modules/Inti/Entities/Jabatan.php

php

<?php

```
namespace Modules\Inti\Entities;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Jabatan extends Model
{
    use HasFactory;

    protected $table = 'master_jabatan';

    protected $fillable = [
        'kode', 'nama', 'level', 'is_struktural',
        'bebas_nilai_kinerja', 'is_active'
    ];

    protected $casts = [
        'is_struktural' => 'boolean',
        'bebas_nilai_kinerja' => 'boolean',
        'is_active' => 'boolean',
    ];

    public function pegawai()
    {
        return $this->hasMany(Pegawai::class, 'jabatan_id');
    }

    public function scopeAktif($query)
    {
        return $query->where('is_active', true);
    }
}
```

Modules/Inti/Entities/Pegawai.php

php

<?php

namespace Modules\Inti\Entities;

use Illuminate\Foundation\Auth\User as Authenticatable;

use Illuminate\Database\Eloquent\Factories\HasFactory;

use Illuminate\Database\Eloquent\SoftDeletes;

use Illuminate\Notifications\Notifiable;

class Pegawai extends Authenticatable

{

use HasFactory, Notifiable, SoftDeletes;

protected \$table = 'master_pegawai';

protected \$fillable = [
 'nomor_identitas', 'tipe_identitas', 'nama', 'jabatan_id', 'bidang_id',
 'email', 'password', 'no_telepon', 'pangkat', 'golongan',
 'gelar_depan', 'gelar_belakang', 'tanggal_lahir', 'jenis_kelamin',
 'alamat', 'status_kepegawaian', 'status_aktif', 'tanggal_masuk',
 'tanggal_keluar', 'atasan_langsung_id', 'foto_profile_path'

];

protected \$hidden = ['password', 'remember_token'];

protected \$casts = [
 'tanggal_lahir' => 'date',
 'tanggal_masuk' => 'date',
 'tanggal_keluar' => 'date',
 'last_login_at' => 'datetime',

];

public function jabatan()
{
 return \$this->belongsTo(Jabatan::class);
}

public function bidang()
{
 return \$this->belongsTo(Bidang::class);
}

public function atasanLangsung()
{
 return \$this->belongsTo(Pegawai::class, 'atasan_langsung_id');
}

public function bawahan()
{
 return \$this->hasMany(Pegawai::class, 'atasan_langsung_id');
}

public function ttdDigital()
{
 return \$this->hasMany(TtdDigital::class);
}


```
public function getNamaLengkapAttribute()
{
    $nama = $this->nama;
    if ($this->gelar_depan) $nama = $this->gelar_depan . ' ' . $nama;
    if ($this->gelar_belakang) $nama .= ', ' . $this->gelar_belakang;
    return $nama;
}
}
```

Buat Seeders

bash

```
php artisan module:make-seeder JabatanSeeder Inti
php artisan module:make-seeder BidangSeeder Inti
php artisan module:make-seeder PegawaiSeeder Inti
```

JabatanSeeder.php

php

`<?php`

`namespace Modules\Inti\Database\Seeders;`

`use Illuminate\Database\Seeder;`

`use Modules\Inti\Entities\Jabatan;`

`class JabatanSeeder extends Seeder`

`{`

`public function run(): void`

`{`

`$jabatan = [`

`['kode' => 'KB', 'nama' => 'Kepala Badan', 'level' => 1, 'is_struktural' => true],`

`['kode' => 'SB', 'nama' => 'Sekretaris Badan', 'level' => 2, 'is_struktural' => true],`

`['kode' => 'KBD', 'nama' => 'Kepala Bidang', 'level' => 3, 'is_struktural' => true],`

`['kode' => 'STF', 'nama' => 'Staff', 'level' => 4, 'is_struktural' => true],`

`['kode' => 'JF', 'nama' => 'Jabatan Fungsional', 'level' => 5, 'is_struktural' => false],`

`['kode' => 'TT', 'nama' => 'Tenaga Teknis', 'level' => 6, 'is_struktural' => false, 'bebas_nilai_kinerja' => true],`

`];`

`foreach ($jabatan as $j) {`

`Jabatan::create($j);`

`}`

`}`

`}`

BidangSeeder.php

php

```
<?php

namespace Modules\Inti\Database\Seeders;

use Illuminate\Database\Seeder;
use Modules\Inti\Entities\Bidang;

class BidangSeeder extends Seeder
{
    public function run(): void
    {
        $bidang = [
            ['kode' => 'SEKRET', 'nama' => 'Sekretariat', 'deskripsi' => 'Koordinasi pelaksanaan tugas', 'warna' => '#2563eb'],
            ['kode' => 'PLAN', 'nama' => 'Perencanaan Pembangunan', 'deskripsi' => 'Perumusan kebijakan perencanaan', 'warna' => '#2563eb'],
            ['kode' => 'EVAL', 'nama' => 'Evaluasi dan Pengendalian', 'deskripsi' => 'Evaluasi pembangunan daerah', 'warna' => '#2563eb'],
            ['kode' => 'DATA', 'nama' => 'Data dan Statistik', 'deskripsi' => 'Pengelolaan data pembangunan', 'warna' => '#2563eb'],
        ];

        foreach ($bidang as $b) {
            Bidang::create($b);
        }
    }
}
```

IntiDatabaseSeeder.php

```
php

<?php

namespace Modules\Inti\Database\Seeders;

use Illuminate\Database\Seeder;

class IntiDatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            JabatanSeeder::class,
            BidangSeeder::class,
            PegawaiSeeder::class,
        ]);
    }
}
```

Jalankan Migration

```
bash

php artisan module:migrate Inti --seed
```

Priority: HIGH ★★★★★

Estimasi: 2-3 hari

Tabel: 8 (doc_folder, doc_kategori, doc_jenis, doc_dokumen, doc_file, doc_metadata, doc_log, doc_nomor_counter)

Generate Modul

```
bash

php artisan module:make Dokumen
```

Buat Migration

```
bash

php artisan module:make-migration create_doc_folder_table Dokumen
php artisan module:make-migration create_doc_kategori_table Dokumen
php artisan module:make-migration create_doc_jenis_table Dokumen
php artisan module:make-migration create_doc_dokumen_table Dokumen
php artisan module:make-migration create_doc_file_table Dokumen
php artisan module:make-migration create_doc_metadata_table Dokumen
php artisan module:make-migration create_doc_log_table Dokumen
php artisan module:make-migration create_doc_nomor_counter_table Dokumen
```

Migration Files

create_doc_kategori_table.php

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_kategori', function (Blueprint $table) {
            $table->id();
            $table->string('nama', 50);
            $table->string('icon', 50)->nullable();
            $table->string('warna', 7)->default('#2563eb');
            $table->integer('urutan')->default(0);
            $table->timestamps();

            $table->index('urutan');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_kategori');
    }
};
```

create_doc_jenis_table.php

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_jenis', function (Blueprint $table) {
            $table->id();
            $table->foreignId('kategori_id')->constrained('doc_kategori')->cascadeOnDelete();
            $table->string('kode', 20)->unique();
            $table->string('nama', 100);
            $table->string('folder_pattern')->default('/{bidang}/{jenis}/{year}/{month}/');
            $table->string('nomor_format')->nullable();
            $table->string('allowed_ext')->default('pdf,docx,xlsx');
            $table->integer('max_size_mb')->default(10);
            $table->boolean('perlu_nomor')->default(false);
            $table->timestamps();

            $table->index(['kode', 'kategori_id']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_jenis');
    }
};
```

create_doc_folder_table.php

```
php
```

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_folder', function (Blueprint $table) {
            $table->id();
            $table->foreignId('parent_id')->nullable()->constrained('doc_folder')->onDelete('cascade');
            $table->foreignId('bidang_id')->nullable()->constrained('master_bidang')->onDelete('cascade');
            $table->string('nama', 100);
            $table->string('path')->unique();
            $table->integer('level')->default(0);
            $table->boolean('is_auto')->default(false);
            $table->integer('total_files')->default(0);
            $table->foreignId('created_by')->constrained('master_pegawai')->onDelete('cascade');
            $table->timestamps();

            $table->index(['parent_id', 'bidang_id', 'level']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_folder');
    }
};
```

create_doc_dokumen_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_dokumen', function (Blueprint $table) {
            $table->id();
            $table->string('nomor')->unique()->nullable();
            $table->foreignId('folder_id')->constrained('doc_folder')->cascadeOnDelete();
            $table->foreignId('jenis_id')->constrained('doc_jenis')->restrictOnDelete();
            $table->string('judul');
            $table->text('deskripsi')->nullable();
            $table->date('tanggal_dokumen');
            $table->string('nomor_surat')->nullable();
            $table->enum('status', ['Draft', 'Final', 'Archived'])->default('Draft');
            $table->integer('version')->default(1);
            $table->integer('views')->default(0);
            $table->integer('downloads')->default(0);
            $table->foreignId('uploaded_by')->constrained('master_pegawai')->restrictOnDelete();
            $table->timestamps();

            $table->index(['nomor', 'jenis_id', 'status']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_dokumen');
    }
};
```

create_doc_file_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_file', function (Blueprint $table) {
            $table->id();
            $table->foreignId('dokumen_id')->constrained('doc_dokumen')->cascadeOnDelete();
            $table->integer('version')->default(1);
            $table->string('nama_file');
            $table->string('file_path');
            $table->string('extension', 10);
            $table->integer('size_kb');
            $table->string('hash', 64);
            $table->text('keterangan')->nullable();
            $table->boolean('is_current')->default(true);
            $table->foreignId('uploaded_by')->constrained('master_pegawai')->restrictOnDelete();
            $table->timestamps();

            $table->index(['dokumen_id', 'version', 'is_current']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_file');
    }
};
```

create_doc_metadata_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_metadata', function (Blueprint $table) {
            $table->id();
            $table->foreignId('dokumen_id')->constrained('doc_dokumen')->cascadeOnDelete();
            $table->string('key', 100);
            $table->text('value');
            $table->timestamps();

            $table->index(['dokumen_id', 'key']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_metadata');
    }
};
```

create_doc_log_table.php

php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_log', function (Blueprint $table) {
            $table->id();
            $table->foreignId('dokumen_id')->constrained('doc_dokumen')->cascadeOnDelete();
            $table->foreignId('user_id')->constrained('master_pegawai')->cascadeOnDelete();
            $table->enum('action', ['View', 'Download', 'Upload', 'Edit', 'Delete']);
            $table->string('ip_address', 45);
            $table->timestamps();

            $table->index(['dokumen_id', 'user_id', 'action']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_log');
    }
};
```

create_doc_nomor_counter_table.php

php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('doc_nomor_counter', function (Blueprint $table) {
            $table->id();
            $table->foreignId('jenis_id')->constrained('doc_jenis')->cascadeOnDelete();
            $table->foreignId('bidang_id')->constrained('master_bidang')->cascadeOnDelete();
            $table->integer('tahun');
            $table->integer('counter')->default(0);
            $table->timestamps();

            $table->unique(['jenis_id', 'bidang_id', 'tahun']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('doc_nomor_counter');
    }
};
```

Buat Models

```
bash

php artisan module:make-model Kategori Dokumen
php artisan module:make-model Jenis Dokumen
php artisan module:make-model Folder Dokumen
php artisan module:make-model Dokumen Dokumen
php artisan module:make-model File Dokumen
```

Jalankan Migration

```
bash

php artisan module:migrate Dokumen
```

 MODUL 3: PERJANJIAN KINERJA

Priority: HIGH    

Estimasi: 4-6 hari

Tabel: 9

Generate Modul

```
bash
```

```
php artisan module:make PerjanjianKinerja
```

Buat Migration

```
bash
```

```
php artisan module:make-migration create_pk_template_table PerjanjianKinerja
php artisan module:make-migration create_pk_template_section_table PerjanjianKinerja
php artisan module:make-migration create_pk_perjanjian_kinerja_table PerjanjianKinerja
php artisan module:make-migration create_pk_dokumen_table PerjanjianKinerja
php artisan module:make-migration create_pk_sasaran_table PerjanjianKinerja
php artisan module:make-migration create_pk_indikator_table PerjanjianKinerja
php artisan module:make-migration create_pk_program_table PerjanjianKinerja
php artisan module:make-migration create_pk_kegiatan_table PerjanjianKinerja
php artisan module:make-migration create_pk_sub_kegiatan_table PerjanjianKinerja
```

Migration Files (Contoh Utama)

`create_pk_template_table.php`

```
php
```

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('pk_template', function (Blueprint $table) {
            $table->id();
            $table->string('kode_template', 50)->unique();
            $table->string('nama_template', 100);
            $table->foreignId('jabatan_id')->constrained('master_jabatan')->restrictOnDelete();
            $table->integer('tahun');
            $table->text('kop_surat_html')->nullable();
            $table->text('header_template')->nullable();
            $table->text('pernyataan_pembuka')->nullable();
            $table->text('pernyataan_penutup')->nullable();
            $table->text('footer_template')->nullable();
            $table->enum('page_size', ['A4', 'Legal', 'Letter'])->default('A4');
            $table->enum('orientation', ['Portrait', 'Landscape'])->default('Portrait');
            $table->integer('versi')->default(1);
            $table->boolean('is_active')->default(true);
            $table->timestamps();

            $table->index(['jabatan_id', 'tahun', 'is_active']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('pk_template');
    }
};
```

create_pk_perjanjian_kinerja_table.php

php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('pk_perjanjian_kinerja', function (Blueprint $table) {
            $table->id();
            $table->string('nomor_perjanjian', 50)->unique();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->foreignId('atasan_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->foreignId('template_id')->constrained('pk_template')->restrictOnDelete();
            $table->integer('tahun');
            $table->date('periode_mulai');
            $table->date('periode_selesai');
            $table->string('tempat_ttd', 50)->default('Kuningan');
            $table->date('tanggal_ttd')->nullable();
            $table->decimal('total_anggaran', 15, 2)->default(0);
            $table->enum('status_dokumen', ['Draft', 'Generated', 'Menunggu_TTD', 'Aktif', 'Selesai', 'Dibatalkan'])->default('Draft');
            $table->text('catatan')->nullable();
            $table->boolean('is_locked')->default(false);
            $table->timestamps();
            $table->softDeletes();

            $table->index(['pegawai_id', 'tahun', 'status_dokumen']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('pk_perjanjian_kinerja');
    }
};
```

create_pk_sasaran_table.php

```
php
```

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('pk_sasaran', function (Blueprint $table) {
            $table->id();
            $table->foreignId('perjanjian_kinerja_id')->constrained('pk_perjanjian_kinerja')->cascadeOnDelete();
            $table->integer('urutan');
            $table->text('sasaran_strategis');
            $table->timestamps();

            $table->index('perjanjian_kinerja_id');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('pk_sasaran');
    }
};
```

Jalankan Migration

```
bash

php artisan module:migrate PerjanjianKinerja
```

 MODUL 4: PENUGASAN

Priority: HIGH    

Estimasi: 5-7 hari

Tabel: 9

Generate Modul

```
bash

php artisan module:make Penugasan
```

Buat Migration

```
bash
```

php artisan module:make-migration create_knj_tugas_pokok_table Penugasan

php artisan module:make-migration create_knj_indikator_tugas_table Penugasan

php artisan module:make-migration create_knj_tugas_harian_table Penugasan

php artisan module:make-migration create_knj_tugas_tambahan_table Penugasan

php artisan module:make-migration create_knj_penugasan_mandiri_table Penugasan

php artisan module:make-migration create_knj_progress_table Penugasan

php artisan module:make-migration create_knj_foto_bukti_table Penugasan

php artisan module:make-migration create_knj_delegasi_table Penugasan

php artisan module:make-migration create_knj_workload_table Penugasan

Migration Files (Contoh Utama)

create_knj_tugas_pokok_table.php

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('knj_tugas_pokok', function (Blueprint $table) {
            $table->id();
            $table->foreignId('perjanjian_kinerja_id')->constrained('pk_perjanjian_kinerja')->restrictOnDelete();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->foreignId('pemberi_tugas_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->string('nama_tugas');
            $table->text('deskripsi');
            $table->decimal('bobot_persen', 5, 2);
            $table->date('periode_mulai');
            $table->date('periode_selesai');
            $table->decimal('target_value', 10, 2);
            $table->string('satuan', 50);
            $table->enum('status', ['Pending', 'Diterima', 'Dikerjakan', 'Selesai', 'Tidak_Selesai', 'Divalidasi'])->default('Pending');
            $table->decimal('progress_persen', 5, 2)->default(0);
            $table->timestamp('diterima_at')->nullable();
            $table->foreignId('dokumen_lampiran_id')->nullable()->constrained('doc_dokumen')->nullOnDelete();
            $table->timestamps();
            $table->softDeletes();

            $table->index(['pegawai_id', 'status']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('knj_tugas_pokok');
    }
};
```

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('knj_progress', function (Blueprint $table) {
            $table->id();
            $table->foreignId('tugas_pokok_id')->nullable()->constrained('knj_tugas_pokok')->cascadeOnDelete();
            $table->foreignId('tugas_harian_id')->nullable()->constrained('knj_tugas_harian')->cascadeOnDelete();
            $table->foreignId('tugas_tambahan_id')->nullable()->constrained('knj_tugas_tambahan')->cascadeOnDelete();
            $table->foreignId('penugasan_mandiri_id')->nullable()->constrained('knj_penugasan_mandiri')->cascadeOnDelete();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->date('tanggal');
            $table->decimal('progress_persen', 5, 2);
            $table->text('deskripsi_kegiatan');
            $table->text('kendala')->nullable();
            $table->foreignId('dokumen_bukti_id')->nullable()->constrained('doc_dokumen')->nullOnDelete();
            $table->timestamps();

            $table->index(['pegawai_id', 'tanggal']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('knj_progress');
    }
};
```

Jalankan Migration

```
bash

php artisan module:migrate Penugasan
```

MODUL 5: EVALUASI

Priority: HIGH    

Estimasi: 4-5 hari

Tabel: 4

Generate Modul

```
bash
```


Buat Migration

```
bash

php artisan module:make-migration create_knj_validasi_table Evaluasi
php artisan module:make-migration create_knj_revisi_table Evaluasi
php artisan module:make-migration create_knj_nilai_bulanan_table Evaluasi
php artisan module:make-migration create_knj_nilai_tahunan_table Evaluasi
```

Migration Files

create_knj_validasi_table.php

```
php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('knj_validasi', function (Blueprint $table) {
            $table->id();
            $table->foreignId('tugas_pokok_id')->nullable()->constrained('knj_tugas_pokok')->cascadeOnDelete();
            $table->foreignId('tugas_harian_id')->nullable()->constrained('knj_tugas_harian')->cascadeOnDelete();
            $table->foreignId('tugas_tambahan_id')->nullable()->constrained('knj_tugas_tambahan')->cascadeOnDelete();
            $table->foreignId('penugasan_mandiri_id')->nullable()->constrained('knj_penugasan_mandiri')->cascadeOnDelete();
            $table->foreignId('validator_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->enum('hasil_validasi', ['Terima', 'Minta_Revisi', 'Tolak']);
            $table->text('catatan')->nullable();
            $table->integer('penilaian_kualitas')->nullable();
            $table->boolean('kesesuaian_target')->default(false);
            $table->decimal('nilai_akhir', 5, 2)->nullable();
            $table->date('tanggal_validasi');
            $table->timestamps();

            $table->index(['validator_id', 'tanggal_validasi']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('knj_validasi');
    }
};
```

create_knj_nilai_bulanan_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('knj_nilai_bulanan', function (Blueprint $table) {
            $table->id();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->restrictOnDelete();
            $table->integer('tahun');
            $table->integer('bulan');
            $table->decimal('nilai_tugas_pokok', 5, 2)->default(0);
            $table->decimal('nilai_tugas_harian', 5, 2)->default(0);
            $table->decimal('nilai_tugas_tambahan', 5, 2)->default(0);
            $table->decimal('total_penalti', 5, 2)->default(0);
            $table->decimal('total_bonus', 5, 2)->default(0);
            $table->decimal('nilai_total', 5, 2)->default(0);
            $table->enum('kategori_nilai', ['Sangat_Baik', 'Baik', 'Cukup', 'Kurang', 'Sangat_Kurang'])->nullable();
            $table->boolean('is_approved')->default(false);
            $table->foreignId('approved_by')->nullable()->constrained('master_pegawai')->nullOnDelete();
            $table->text('catatan_atasan')->nullable();
            $table->timestamp('approved_at')->nullable();
            $table->timestamps();

            $table->unique(['pegawai_id', 'tahun', 'bulan']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('knj_nilai_bulanan');
    }
};
```

Jalankan Migration

```
bash

php artisan module:migrate Evaluasi
```

 MODUL 6: SISTEM

Priority: MEDIUM   

Estimasi: 2-3 hari

Tabel: 4

Generate Modul

```
bash
```

```
php artisan module:make Sistem
```

Buat Migration

```
bash
```

```
php artisan module:make-migration create_sys_notifikasi_table Sistem
php artisan module:make-migration create_sys_preferensi_notifikasi_table Sistem
php artisan module:make-migration create_sys_audit_log_table Sistem
php artisan module:make-migration create_sys_config_table Sistem
```

Migration Files

create_sys_notifikasi_table.php

```
php
```

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('sys_notifikasi', function (Blueprint $table) {
            $table->id();
            $table->foreignId('pegawai_id')->constrained('master_pegawai')->cascadeOnDelete();
            $table->string('judul');
            $table->text('pesan');
            $table->enum('tipe', ['Tugas_Baru', 'Approval', 'Revisi', 'Validasi', 'Alert', 'Reminder', 'Nilai', 'Dokumen']);
            $table->string('entity_type', 50)->nullable();
            $table->unsignedBigInteger('entity_id')->nullable();
            $table->string('action_url')->nullable();
            $table->boolean('is_read')->default(false);
            $table->timestamp('read_at')->nullable();
            $table->timestamps();

            $table->index(['pegawai_id', 'is_read', 'tipe']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sys_notifikasi');
    }
};
```

create_sys_audit_log_table.php

```
php
```

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
    public function up(): void
    {
        Schema::create('sys_audit_log', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->nullable()->constrained('master_pegawai')->onDelete('cascade');
            $table->string('table_name', 100);
            $table->unsignedBigInteger('record_id')->nullable();
            $table->enum('action', ['CREATE', 'UPDATE', 'DELETE', 'LOGIN', 'LOGOUT', 'EXPORT']);
            $table->json('old_values')->nullable();
            $table->json('new_values')->nullable();
            $table->string('ip_address', 45);
            $table->string('user_agent')->nullable();
            $table->text('description')->nullable();
            $table->timestamps();

            $table->index(['user_id', 'action', 'table_name']);
            $table->index('created_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sys_audit_log');
    }
};
```

create_sys_config_table.php

php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('sys_config', function (Blueprint $table) {
            $table->id();
            $table->string('config_key', 100)->unique();
            $table->string('config_group', 50);
            $table->text('config_value');
            $table->enum('value_type', ['string', 'integer', 'boolean', 'json'])->default('string');
            $table->text('description')->nullable();
            $table->boolean('is_public')->default(false);
            $table->timestamps();

            $table->index(['config_key', 'config_group']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sys_config');
    }
};
```

Jalankan Migration

```
bash

php artisan module:migrate Sistem
```

 MODUL 7: DASHBOARD

Priority: MEDIUM   

Estimasi: 2-3 hari

Tabel: 0 (Read-only)

Generate Modul

```
bash

php artisan module:make Dashboard
```

Buat Controller

```
bash

php artisan module:make-controller DashboardController Dashboard
```

php

<?php

```
namespace Modules\Dashboard\Http\Controllers;

use Illuminate\Routing\Controller;
use Modules\Dokumen\Entities\Dokumen;
use Modules\Dokumen\Entities\Folder;
use Modules\Inti\Entities\Pegawai;
use Illuminate\Support\Facades\DB;

class DashboardController extends Controller
{
    public function index()
    {
        $stats = [
            'total_dokumen' => Dokumen::count(),
            'total_folder' => Folder::count(),
            'total_pegawai' => Pegawai::aktif()->count(),
            'storage_terpakai' => $this->getStorageUsed(),
        ];

        $dokumenPerKategori = DB::table('doc_dokumen')
            ->join('doc_jenis', 'doc_dokumen.jenis_id', '=', 'doc_jenis.id')
            ->join('doc_kategori', 'doc_jenis.kategori_id', '=', 'doc_kategori.id')
            ->select('doc_kategori.nama', DB::raw('count(*) as total'))
            ->groupBy('doc_kategori.id', 'doc_kategori.nama')
            ->get();

        $aktivitasTerbaru = DB::table('doc_log')
            ->join('doc_dokumen', 'doc_log.dokumen_id', '=', 'doc_dokumen.id')
            ->join('master_pegawai', 'doc_log.user_id', '=', 'master_pegawai.id')
            ->select('doc_log.*', 'doc_dokumen.judul', 'master_pegawai.nama')
            ->orderBy('doc_log.created_at', 'desc')
            ->limit(10)
            ->get();

        return view('dashboard::index', compact('stats', 'dokumenPerKategori', 'aktivitasTerbaru'));
    }

    private function getStorageUsed()
    {
        $totalSize = DB::table('doc_file')->sum('size_kb');
        return $totalSize; // KB
    }
}
```

Buat View

blade

@extends('layouts.app')

@section('content')

<div class="container-fluid">

<h2 class="mb-4">Dashboard TERDEPAN</h2>

<!-- Stats Cards -->

<div class="row mb-4">

<div class="col-md-3">

<div class="card">

<div class="card-body">

<h5>Total Dokumen</h5>

<h2>{{ \$stats['total_dokumen'] }}</h2>

</div>

</div>

</div>

<div class="col-md-3">

<div class="card">

<div class="card-body">

<h5>Total Folder</h5>

<h2>{{ \$stats['total_folder'] }}</h2>

</div>

</div>

</div>

<div class="col-md-3">

<div class="card">

<div class="card-body">

<h5>Total Pengguna</h5>

<h2>{{ \$stats['total_pegawai'] }}</h2>

</div>

</div>

</div>

<div class="col-md-3">

<div class="card">

<div class="card-body">

<h5>Storage Terpakai</h5>

<h2>{{ number_format(\$stats['storage_terpakai'] / 1024, 2) }} MB</h2>

</div>

</div>

</div>

</div>

<!-- Charts & Tables -->

<div class="row">

<div class="col-md-6">

<div class="card">

<div class="card-header">

<h5>Dokumen Per Kategori</h5>

</div>

<div class="card-body">

<canvas id="dokumenChart"></canvas>

</div>

</div>

</div>

<div class="col-md-6">

<div class="card">

```

<div class="card-header">
  <h5>Aktivitas Terbaru</h5>
</div>
<div class="card-body">
  <table class="table">
    <thead>
      <tr>
        <th>Waktu</th>
        <th>Pengguna</th>
        <th>Aksi</th>
        <th>Dokumen</th>
      </tr>
    </thead>
    <tbody>
      @foreach($SaktivitasTerbaru as $Saktivitas)
      <tr>
        <td>{{ $Saktivitas->created_at }}</td>
        <td>{{ $Saktivitas->nama }}</td>
        <td>{{ $Saktivitas->action }}</td>
        <td>{{ $Saktivitas->judul }}</td>
      </tr>
      @endforeach
    </tbody>
  </table>
</div>
</div>
</div>
</div>
</div>
@endsection

```

Setup Routes

Modules/Dashboard/Routes/web.php

```
php

<?php

use Illuminate\Support\Facades\Route;
use Modules\Dashboard\Http\Controllers\DashboardController;

Route::get('/', [DashboardController::class, 'index'])->name('dashboard');
```

TESTING & DEPLOYMENT

1. Unit Testing


```
# Generate test untuk setiap modul

php artisan module:make-test JabatanTest Inti --unit
php artisan module:make-test DokumenTest Dokumen --unit


# Run test per modul

php artisan test --testsuite=Inti
php artisan test --testsuite=Dokumen
```

Contoh Unit Test:

```
php

<?php

namespace Modules\Inti\Tests\Unit;

use Tests\TestCase;
use Modules\Inti\Entities\Pegawai;
use Modules\Inti\Entities\Jabatan;
use Illuminate\Foundation\Testing\RefreshDatabase;

class PegawaiTest extends TestCase
{
    use RefreshDatabase;

    public function test_pegawai_dapat_dibuat()
    {
        $jabatan = Jabatan::factory()->create();
        $pegawai = Pegawai::factory()->create(['jabatan_id' => $jabatan->id]);

        $this->assertDatabaseHas('master_pegawai', [
            'email' => $pegawai->email
        ]);
    }

    public function test_nama_lengkap_dengan_gelar()
    {
        $pegawai = Pegawai::factory()->create([
            'nama' => 'Ahmad',
            'gelar_depan' => 'Dr.',
            'gelar_belakang' => 'M.Si.'
        ]);

        $this->assertEquals('Dr. Ahmad, M.Si.', $pegawai->nama_lengkap);
    }
}
```

2. Feature Testing

```
bash

php artisan module:make-test JabatanFeatureTest Inti
```

Contoh Feature Test:

php

<?php

```
namespace Modules\Inti\Tests\Feature;

use Tests\TestCase;
use Modules\Inti\Entities\Jabatan;
use Modules\Inti\Entities\Pegawai;
use Illuminate\Foundation\Testing\RefreshDatabase;

class JabatanControllerTest extends TestCase
{
    use RefreshDatabase;

    public function test_dapat_melihat_daftar_jabatan()
    {
        $user = Pegawai::factory()->create();

        $response = $this->actingAs($user)->get(route('inti.jabatan.index'));

        $response->assertStatus(200);
        $response->assertViewIs('inti::jabatan.index');
    }

    public function test_dapat_membuat_jabatan_baru()
    {
        $user = Pegawai::factory()->create();

        $data = [
            'kode' => 'TEST',
            'nama' => 'Jabatan Test',
            'level' => 5,
            'is_struktural' => true
        ];

        $response = $this->actingAs($user)
            ->post(route('inti.jabatan.store'), $data);

        $response->assertRedirect(route('inti.jabatan.index'));
        $this->assertDatabaseHas('master_jabatan', ['kode' => 'TEST']);
    }
}
```

3. Migration Testing

bash

```
# Test rollback semua modul
php artisan module:migrate-rollback

# Test fresh migration dengan seed
php artisan module:migrate-fresh --seed

# Test per modul
php artisan module:migrate-refresh Inti --seed
php artisan module:migrate-refresh Dokumen --seed
```

4. Database Seeding

```
bash

# Seed semua modul sekaligus
php artisan db:seed

# Seed per modul
php artisan module:seed Inti
php artisan module:seed Dokumen
php artisan module:seed PerjanjianKinerja
```

5. Performance Testing

```
bash

# Install Laravel Debugbar
composer require barryvdh/laravel-debugbar --dev

# Optimize aplikasi
php artisan optimize
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

6. Code Quality

```
bash

# Install PHP CS Fixer
composer require --dev friendsofphp/php-cs-fixer

# Fix coding standards
vendor/bin/php-cs-fixer fix Modules/

# Install PHPStan
composer require --dev phpstan/phpstan

# Run static analysis
vendor/bin/phpstan analyse Modules/
```

7. Deployment Checklist

Pre-Deployment:

bash

1. Update dependencies

`composer install --no-dev --optimize-autoloader`

2. Generate key

`php artisan key:generate`

3. Run migrations

`php artisan migrate --force`

4. Clear & cache

`php artisan config:cache`

`php artisan route:cache`

`php artisan view:cache`

5. Set permissions

`chmod -R 755 storage bootstrap/cache`

.env

Production:

env

APP_ENV=production

APP_DEBUG=false

APP_URL=https://terdepan.bappeda.kuningankab.go.id

DB_CONNECTION=mysql

DB_HOST=localhost

DB_PORT=3306

DB_DATABASE=terdepan_production

DB_USERNAME=terdepan_user

DB_PASSWORD=strong_password_here

CACHE_DRIVER=redis

SESSION_DRIVER=redis

QUEUE_CONNECTION=redis

REDIS_HOST=127.0.0.1

REDIS_PASSWORD=null

REDIS_PORT=6379

Nginx Configuration:

nginx

```
server {  
    listen 80;  
    listen [::]:80;  
    server_name terdepan.bappeda.kuningankab.go.id;  
    root /var/www/terdepan/public;  
  
    add_header X-Frame-Options "SAMEORIGIN";  
    add_header X-Content-Type-Options "nosniff";  
  
    index index.php;  
  
    charset utf-8;  
  
    location / {  
        try_files $uri $uri/ /index.php?$query_string;  
    }  
  
    location = /favicon.ico { access_log off; log_not_found off; }  
    location = /robots.txt  { access_log off; log_not_found off; }  
  
    error_page 404 /index.php;  
  
    location ~ \.php$ {  
        fastcgi_pass unix:/var/run/php/php8.2-fpm.sock;  
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;  
        include fastcgi_params;  
    }  
  
    location ~ /\.(!well-known).* {  
        deny all;  
    }  
}
```

8. Monitoring & Logging

Install Laravel Telescope:

```
bash  
  
composer require laravel/telescope --dev  
php artisan telescope:install  
php artisan migrate
```

Setup Log Rotation:

```
bash
```

```
# /etc/logrotate.d/laravel
/var/www/terdepan/storage/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data www-data
    sharedscripts
}
```

9. Backup Strategy

```
bash

# Install Laravel Backup
composer require spatie/laravel-backup

# Publish config
php artisan vendor:publish --provider="Spatie\Backup\BackupServiceProvider"

# Setup cron
# * * * * * php /var/www/terdepan/artisan schedule:run >> /dev/null 2>&1
```

config/backup.php:

```
php

'backup' => [
    'name' => 'terdepan',
    'source' => [
        'files' => [
            'include' => [
                base_path(),
            ],
            'exclude' => [
                base_path('vendor'),
                base_path('node_modules'),
            ],
        ],
        'databases' => ['mysql'],
    ],
    'destination' => [
        'disks' => ['local', 's3'],
    ],
],
```

10. Security Checklist

- ☒ HTTPS/SSL enabled
- ☒ CSRF protection active
- ☒ SQL injection prevention (Eloquent ORM)
- ☒ XSS protection (Blade escaping)
- ☒ Password hashing (bcrypt)
- ☒ Rate limiting configured

- ✓ File upload validation
- ✓ Environment variables secured
- ✓ Database credentials encrypted
- ✓ Admin access restricted

MONITORING COMMANDS

```
bash

# Check application status
php artisan about

# Monitor queue workers
php artisan queue:monitor

# Check failed jobs
php artisan queue:failed

# Monitor routes
php artisan route:list

# Database status
php artisan db:show

# Clear all cache
php artisan optimize:clear
```

KESIMPULAN

Tutorial lengkap ini mencakup:

✓ 7 Modul Lengkap:

1. **Inti** - Master data (Jabatan, Bidang, Pegawai, TTD Digital)
2. **Dokumen** - Manajemen dokumen & file storage
3. **Perjanjian Kinerja** - Template & perjanjian kinerja pegawai
4. **Penugasan** - Tugas pokok, harian, tambahan, mandiri
5. **Evaluasi** - Validasi, revisi, penilaian kinerja
6. **Sistem** - Notifikasi, audit log, konfigurasi
7. **Dashboard** - Visualisasi data & statistik

✓ 32 Tabel Database:

- Semua tabel dengan foreign keys
- Indexes untuk performance
- Soft deletes untuk data integrity
- Timestamps untuk audit trail

✓ Best Practices:

- Modular architecture

Module Commands


```
# Generate new module
php artisan module:make ModuleName

# Generate migration
php artisan module:make-migration create_table_name ModuleName

# Generate model
php artisan module:make-model ModelName ModuleName

# Generate controller
php artisan module:make-controller ControllerName ModuleName

# Generate seeder
php artisan module:make-seeder SeederName ModuleName

# Generate request
php artisan module:make-request RequestName ModuleName

# Generate resource
php artisan module:make-resource ResourceName ModuleName

# Generate test
php artisan module:make-test TestName ModuleName

# Migrate specific module
php artisan module:migrate ModuleName

# Seed specific module
php artisan module:seed ModuleName

# Rollback specific module
php artisan module:migrate-rollback ModuleName

# Refresh specific module
php artisan module:migrate-refresh ModuleName

# Enable module
php artisan module:enable ModuleName

# Disable module
php artisan module:disable ModuleName

# List all modules
php artisan module:list

# Update module
php artisan module:update ModuleName
```

Database Commands

```
bash
```

Create migration

php artisan make:migration create_table_name

Run migrations

php artisan migrate

Rollback migrations

php artisan migrate:rollback

Rollback all migrations

php artisan migrate:reset

Fresh migration (drop all tables)

php artisan migrate:fresh

Fresh migration with seed

php artisan migrate:fresh --seed

Show migration status

php artisan migrate:status

Create seeder

php artisan make:seeder SeederName

Run seeders

php artisan db:seed

Run specific seeder

php artisan db:seed --class=SeederName

Show database info

php artisan db:show

Monitor database

php artisan db:monitor

Cache Commands

bash

| |
|--|
| <div># <i>Clear application cache</i>
php artisan cache:clear</div> |
| <div># <i>Clear config cache</i>
php artisan config:clear</div> |
| <div># <i>Clear route cache</i>
php artisan route:clear</div> |
| <div># <i>Clear view cache</i>
php artisan view:clear</div> |
| <div># <i>Clear compiled files</i>
php artisan clear-compiled</div> |
| <div># <i>Cache config</i>
php artisan config:cache</div> |
| <div># <i>Cache routes</i>
php artisan route:cache</div> |
| <div># <i>Cache views</i>
php artisan view:cache</div> |
| <div># <i>Optimize application</i>
php artisan optimize</div> |
| <div># <i>Clear all optimizations</i>
php artisan optimize:clear</div> |

APPENDIX B: HELPER FUNCTIONS

Create Helper File

app/Helpers/helpers.php

| |
|----------------|
| <div>php</div> |
|----------------|

<?php

```
use Modules\Inti\Entities\Pegawai;
use Modules\Sistem\Entities\Config;
use Modules\Sistem\Entities\Notifikasi;

/**
 * Get authenticated pegawai
 */
if (!function_exists('pegawai')) {
    function pegawai(): ?Pegawai
    {
        return auth()->user();
    }
}

/**
 * Get config value
 */
if (!function_exists('app_config')) {
    function app_config(string $key, $default = null)
    {
        return Config::getValue($key, $default);
    }
}

/**
 * Send notification to pegawai
 */
if (!function_exists('send_notification')) {
    function send_notification(
        int $pegawaiId,
        string $judul,
        string $pesan,
        string $tipe = 'Alert',
        ?string $actionUrl = null
    ): void {
        Notifikasi::create([
            'pegawai_id' => $pegawaiId,
            'judul' => $judul,
            'pesan' => $pesan,
            'tipe' => $tipe,
            'action_url' => $actionUrl,
        ]);
    }
}

/**
 * Format file size
 */
if (!function_exists('format_file_size')) {
    function format_file_size(int $sizeKb): string
    {
        $units = ['KB', 'MB', 'GB', 'TB'];
        $size = $sizeKb;
        $unitIndex = 0;
```

```
while ($size >= 1024 && $unitIndex < count($units) - 1) {
    $size /= 1024;
    $unitIndex++;
}

return number_format($size, 2) . ' ' . $units[$unitIndex];
}
}

/**
 * Generate nomor dokumen
 */
if (!function_exists('generate_nomor_dokumen')) {
    function generate_nomor_dokumen(
        int $jenisId,
        int $bidangId,
        int $tahun
    ): string {
        $jenis = \Modules\Dokumen\Entities\Jenis::find($jenisId);
        $bidang = \Modules\Inti\Entities\Bidang::find($bidangId);

        $counter = \Modules\Dokumen\Entities\NomorCounter::firstOrCreate(
            [
                'jenis_id' => $jenisId,
                'bidang_id' => $bidangId,
                'tahun' => $tahun,
            ],
            ['counter' => 0]
        );

        $counter->increment('counter');
        $counter->refresh();

        $format = $jenis->nomor_format;
        $nomor = str_replace(
            ['{bidang}', '{year}', '{seq}'],
            [$bidang->kode, $tahun, str_pad($counter->counter, 4, '0', STR_PAD_LEFT)],
            $format
        );

        return $nomor;
    }
}

/**
 * Check if file extension allowed
 */
if (!function_exists('is_allowed_extension')) {
    function is_allowed_extension(string $filename, string $allowedExt): bool
    {
        $ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));
        $allowed = array_map('trim', explode(',', strtolower($allowedExt)));

        return in_array($ext, $allowed);
    }
}
```

```
}

/**
 * Get file hash
 */
if (!function_exists('get_file_hash')) {
    function get_file_hash(string $filePath): string
    {
        return hash_file('sha256', $filePath);
    }
}

/**
 * Format tanggal Indonesia
 */
if (!function_exists('tanggal_indo')) {
    function tanggal_indo($date, $format = 'long'): string
    {
        if (!$date) return '-';

        $date = \Carbon\Carbon::parse($date);

        $bulan = [
            1 => 'Januari', 'Februari', 'Maret', 'April', 'Mei', 'Juni',
            'Juli', 'Agustus', 'September', 'Oktober', 'November', 'Desember'
        ];

        if ($format === 'short') {
            return $date->format('d/m/Y');
        }

        return $date->day . ' ' . $bulan[$date->month] . ' ' . $date->year;
    }
}

/**
 * Get nilai kategori color
 */
if (!function_exists('nilai_color')) {
    function nilai_color(string $kategori): string
    {
        return match($kategori) {
            'Sangat_Baik' => 'success',
            'Baik' => 'primary',
            'Cukup' => 'info',
            'Kurang' => 'warning',
            'Sangat_Kurang' => 'danger',
            default => 'secondary'
        };
    }
}

/**
 * Format nama kategori nilai
 */
if (!function_exists('format_kategori_nilai')) {
```

```
function format_kategori_nilai(string $kategori): string
{
    return str_replace('_', ' ', $kategori);
}
}
```

Register Helpers

composer.json

```
json

{
    "autoload": {
        "files": [
            "app/Helpers/helpers.php"
        ]
    }
}
```

```
bash

# Regenerate autoload
composer dump-autoload
```

APPENDIX C: SERVICE LAYER PATTERN

Create Base Service

app/Services/BaseService.php

```
php
```

```
<?php
```

```
namespace App\Services;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Support\Facades\DB;
```

```
use Exception;
```

```
abstract class BaseService
```

```
{
```

```
    protected $model;
```

```
    public function __construct(Model $model)
```

```
    {
```

```
        $this->model = $model;
```

```
    }
```

```
    public function all()
```

```
    {
```

```
        return $this->model->all();
```

```
    }
```

```
    public function paginate(int $perPage = 15)
```

```
    {
```

```
        return $this->model->paginate($perPage);
```

```
    }
```

```
    public function find(int $id)
```

```
    {
```

```
        return $this->model->find($id);
```

```
    }
```

```
    public function findOrFail(int $id)
```

```
    {
```

```
        return $this->model->findOrFail($id);
```

```
    }
```

```
    public function create(array $data)
```

```
    {
```

```
        DB::beginTransaction();
```

```
        try {
```

```
            $record = $this->model->create($data);
```

```
            DB::commit();
```

```
            return $record;
```

```
        } catch (Exception $e) {
```

```
            DB::rollBack();
```

```
            throw $e;
```

```
        }
```

```
    }
```

```
    public function update(int $id, array $data)
```

```
    {
```

```
        DB::beginTransaction();
```



```
        try {
            $record = $this->findOrFail($id);
            $record->update($data);
            DB::commit();

            return $record;
        } catch (Exception $e) {
            DB::rollBack();
            throw $e;
        }
    }

    public function delete(int $id)
    {
        DB::beginTransaction();

        try {
            $record = $this->findOrFail($id);
            $record->delete();
            DB::commit();

            return true;
        } catch (Exception $e) {
            DB::rollBack();
            throw $e;
        }
    }
}
```

Example Service Implementation

Modules/Dokumen/Services/DokumenService.php

php

```
<?php
```

```
namespace Modules\Dokumen\Services;
```

```
use App\Services\BaseService;
```

```
use Modules\Dokumen\Entities\Dokumen;
```

```
use Modules\Dokumen\Entities\File as DokumenFile;
```

```
use Illuminate\Support\Facades\Storage;
```

```
use Illuminate\Http\UploadedFile;
```

```
class DokumenService extends BaseService
```

```
{
```

```
    public function __construct(Dokumen $model)
```

```
    {
```

```
        parent::__construct($model);
```

```
    }
```

```
    public function uploadDokumen(array $data, UploadedFile $file): Dokumen
```

```
    {
```

```
        \DB::beginTransaction();
```

```
        try {
```

```
            // Generate nomor dokumen jika perlu
```

```
            if ($data['perlu_nomor']) {
```

```
                $data['nomor'] = generate_nomor_dokumen(
```

```
                    $data['jenis_id'],
```

```
                    $data['bidang_id'],
```

```
                    date('Y')
```

```
                );
```

```
            }
```

```
            // Create dokumen record
```

```
            $dokumen = $this->create($data);
```

```
            // Upload file
```

```
            $path = $file->store('dokumen/' . date('Y/m'), 'public');
```

```
            // Create file record
```

```
            DokumenFile::create([
```

```
                'dokumen_id' => $dokumen->id,
```

```
                'version' => 1,
```

```
                'nama_file' => $file->getClientOriginalName(),
```

```
                'file_path' => $path,
```

```
                'extension' => $file->getClientOriginalExtension(),
```

```
                'size_kb' => round($file->getSize() / 1024),
```

```
                'hash' => hash_file('sha256', $file->getRealPath()),
```

```
                'is_current' => true,
```

```
                'uploaded_by' => auth()->id(),
```

```
            ]);
```

```
            \DB::commit();
```

```
            return $dokumen;
```

```
        } catch (\Exception $e) {
```

```
            \DB::rollBack();
```

```
// Delete uploaded file if exists
if (isset($path)) {
    Storage::disk('public')->delete($path);
}

throw $e;
}
}

public function updateVersion(int $dokumenId, UploadedFile $file, string $keterangan = null): DokumenFile
{
    \DB::beginTransaction();

    try {
        $dokumen = $this->findOrFail($dokumenId);

        // Set all files to not current
        DokumenFile::where('dokumen_id', $dokumenId)
            ->update(['is_current' => false]);

        // Get next version
        $version = DokumenFile::where('dokumen_id', $dokumenId)
            ->max('version') + 1;

        // Upload new file
        $path = $file->store('dokumen/' . date('Y/m'), 'public');

        // Create new file record
        $newFile = DokumenFile::create([
            'dokumen_id' => $dokumenId,
            'version' => $version,
            'nama_file' => $file->getClientOriginalName(),
            'file_path' => $path,
            'extension' => $file->getClientOriginalExtension(),
            'size_kb' => round($file->getSize() / 1024),
            'hash' => hash_file('sha256', $file->getRealPath()),
            'keterangan' => $keterangan,
            'is_current' => true,
            'uploaded_by' => auth()->id(),
        ]);

        // Update dokumen version
        $dokumen->update(['version' => $version]);

        \DB::commit();

        return $newFile;
    } catch (\Exception $e) {
        \DB::rollBack();

        if (isset($path)) {
            Storage::disk('public')->delete($path);
        }
    }
}
```

```
        throw $e;
    }
}

public function downloadDokumen(int $dokumenId)
{
    $dokumen = $this->findOrFail($dokumenId);
    $file = $dokumen->currentFile;

    if (!$file) {
        throw new \Exception('File tidak ditemukan');
    }

    // Increment download counter
    $dokumen->increment('downloads');

    // Log activity
    \Modules\Dokumen\Entities\Log::create([
        'dokumen_id' => $dokumenId,
        'user_id' => auth()->id(),
        'action' => 'Download',
        'ip_address' => request()->ip(),
    ]);

    return Storage::disk('public')->download($file->file_path, $file->nama_file);
}
}
```

APPENDIX D: REPOSITORY PATTERN

Create Base Repository

app/Repositories/BaseRepository.php

php

```
<?php
```

```
namespace App\Repositories;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
abstract class BaseRepository
```

```
{
```

```
    protected $model;
```

```
    public function __construct(Model $model)
```

```
    {
```

```
        $this->model = $model;
```

```
    }
```

```
    public function all()
```

```
    {
```

```
        return $this->model->all();
```

```
    }
```

```
    public function find($id)
```

```
    {
```

```
        return $this->model->find($id);
```

```
    }
```

```
    public function findOrFail($id)
```

```
    {
```

```
        return $this->model->findOrFail($id);
```

```
    }
```

```
    public function create(array $attributes)
```

```
    {
```

```
        return $this->model->create($attributes);
```

```
    }
```

```
    public function update($id, array $attributes)
```

```
    {
```

```
        $record = $this->findOrFail($id);
```

```
        $record->update($attributes);
```

```
        return $record;
```

```
    }
```

```
    public function delete($id)
```

```
    {
```

```
        return $this->model->destroy($id);
```

```
    }
```

```
    public function paginate($perPage = 15)
```

```
    {
```

```
        return $this->model->paginate($perPage);
```

```
    }
```

```
    public function where($column, $value)
```

```
    {
```

```
        return $this->model->where($column, $value)->get();
```

```
}  
}
```

Example Repository Implementation

Modules/Inti/Repositories/PegawaiRepository.php

```
php
```

```
<?php

namespace Modules\Inti\Repositories;

use App\Repositories\BaseRepository;
use Modules\Inti\Entities\Pegawai;

class PegawaiRepository extends BaseRepository
{
    public function __construct(Pegawai $model)
    {
        parent::__construct($model);
    }

    public function getAktif()
    {
        return $this->model->aktif()->get();
    }

    public function getPNS()
    {
        return $this->model->pns()->get();
    }

    public function getByBidang(int $bidangId)
    {
        return $this->model->where('bidang_id', $bidangId)->get();
    }

    public function getByJabatan(int $jabatanId)
    {
        return $this->model->where('jabatan_id', $jabatanId)->get();
    }

    public function getBawahan(int $atasanId)
    {
        return $this->model->where('atasan_langsung_id', $atasanId)->get();
    }

    public function search(string $keyword)
    {
        return $this->model
            ->where('nama', 'like', "%{$keyword}%")
            ->orWhere('nomor_identitas', 'like', "%{$keyword}%")
            ->orWhere('email', 'like', "%{$keyword}%")
            ->get();
    }
}
```

APPENDIX E: API DEVELOPMENT

Setup API Authentication

```
bash
```

```
# Install Sanctum (already installed)
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

Create API Controller

Modules/Inti/Http/Controllers/Api/PegawaiApiController.php

php


```
<?php
```

```
namespace Modules\Inti\Http\Controllers\Api;
```

```
use Illuminate\Http\JsonResponse;
```

```
use Illuminate\Routing\Controller;
```

```
use Modules\Inti\Entities\Pegawai;
```

```
use Modules\Inti\Http\Requests\PegawaiRequest;
```

```
use Modules\Inti\Repositories\PegawaiRepository;
```

```
class PegawaiApiController extends Controller
```

```
{
```

```
    protected $repository;
```

```
    public function __construct(PegawaiRepository $repository)
```

```
    {
```

```
        $this->repository = $repository;
```

```
    }
```

```
    public function index(): JsonResponse
```

```
    {
```

```
        $pegawai = $this->repository->all();
```

```
        return response()->json([
```

```
            'success' => true,
```

```
            'data' => $pegawai
```

```
        ]);
```

```
    }
```

```
    public function show(int $id): JsonResponse
```

```
    {
```

```
        try {
```

```
            $pegawai = $this->repository->findOrFail($id);
```

```
            return response()->json([
```

```
                'success' => true,
```

```
                'data' => $pegawai
```

```
            ]);
```

```
        } catch (\Exception $e) {
```

```
            return response()->json([
```

```
                'success' => false,
```

```
                'message' => 'Pegawai tidak ditemukan'
```

```
            ], 404);
```

```
        }
```

```
    }
```

```
    public function store(PegawaiRequest $request): JsonResponse
```

```
    {
```

```
        try {
```

```
            $pegawai = $this->repository->create($request->validated());
```

```
            return response()->json([
```

```
                'success' => true,
```

```
                'message' => 'Pegawai berhasil ditambahkan',
```

```
                'data' => $pegawai
```

```
            ], 201);
```

```
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => 'Gagal menambahkan pegawai',
            'error' => $e->getMessage()
        ], 500);
    }
}

public function update(PegawaiRequest $request, int $id): JsonResponse
{
    try {
        $pegawai = $this->repository->update($id, $request->validated());

        return response()->json([
            'success' => true,
            'message' => 'Pegawai berhasil diupdate',
            'data' => $pegawai
        ]);
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => 'Gagal mengupdate pegawai',
            'error' => $e->getMessage()
        ], 500);
    }
}

public function destroy(int $id): JsonResponse
{
    try {
        $this->repository->delete($id);

        return response()->json([
            'success' => true,
            'message' => 'Pegawai berhasil dihapus'
        ]);
    } catch (\Exception $e) {
        return response()->json([
            'success' => false,
            'message' => 'Gagal menghapus pegawai',
            'error' => $e->getMessage()
        ], 500);
    }
}
}
```

API Routes

Modules/Inti/Routes/api.php

php

```
<?php

use Illuminate\Support\Facades\Route;
use Modules\Inti\Http\Controllers\Api\PegawaiApiController;

Route::middleware('auth:sanctum')->prefix('v1')->group(function () {
    Route::apiResource('pegawai', PegawaiApiController::class);
});
```

API Resource Transformer

Modules/Inti/Transformers/PegawaiResource.php

```
php

<?php

namespace Modules\Inti\Transformers;

use Illuminate\Http\Resources\Json\JsonResource;

class PegawaiResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id' => $this->id,
            'nomor_identitas' => $this->nomor_identitas,
            'nama' => $this->nama,
            'nama_lengkap' => $this->nama_lengkap,
            'email' => $this->email,
            'jabatan' => [
                'id' => $this->jabatan->id,
                'nama' => $this->jabatan->nama,
                'kode' => $this->jabatan->kode,
            ],
            'bidang' => [
                'id' => $this->bidang->id,
                'nama' => $this->bidang->nama,
                'kode' => $this->bidang->kode,
            ],
            'status_aktif' => $this->status_aktif,
            'foto_profile' => $this->foto_profile_path
                ? asset('storage/' . $this->foto_profile_path)
                : null,
            'created_at' => $this->created_at->toISOString(),
            'updated_at' => $this->updated_at->toISOString(),
        ];
    }
}
```

Create Custom Exception Handler

app/Exceptions/Handler.php

```
php

<?php

namespace App\Exceptions;

use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
use Throwable;
use Illuminate\Database\Eloquent\ModelNotFoundException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
use Illuminate\Validation\ValidationException;

class Handler extends ExceptionHandler
{
    public function register(): void
    {
        $this->reportable(function (Throwable $e) {
            //
        });

        $this->renderable(function (ModelNotFoundException $e, $request) {
            if ($request->expectsJson()) {
                return response()->json([
                    'success' => false,
                    'message' => 'Resource not found',
                ], 404);
            }
        });

        $this->renderable(function (NotFoundHttpException $e, $request) {
            if ($request->expectsJson()) {
                return response()->json([
                    'success' => false,
                    'message' => 'Endpoint not found',
                ], 404);
            }
        });

        $this->renderable(function (ValidationException $e, $request) {
            if ($request->expectsJson()) {
                return response()->json([
                    'success' => false,
                    'message' => 'Validation error',
                    'errors' => $e->errors(),
                ], 422);
            }
        });
    }
}
```

APPENDIX G: QUEUE & JOBS

Setup Queue

`.env`

```
env

QUEUE_CONNECTION=database
```

```
bash

# Create jobs table
php artisan queue:table
php artisan migrate

# Create job
php artisan make:job ProcessPerjanjianKinerja
```

Example Job

`app/Jobs/ProcessPerjanjianKinerja.php`

```
php
```

```
<?php
```

```
namespace App\Jobs;
```

```
use Illuminate\Bus\Queueable;
```

```
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
use Illuminate\Foundation\Bus\Dispatchable;
```

```
use Illuminate\Queue\InteractsWithQueue;
```

```
use Illuminate\Queue\SerializesModels;
```

```
use Modules\PerjanjianKinerja\Entities\PerjanjianKinerja;
```

```
use Modules\PerjanjianKinerja\Services\PdfGeneratorService;
```

```
class ProcessPerjanjianKinerja implements ShouldQueue
```

```
{
```

```
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
```

```
    protected $perjanjianKinerjaId;
```

```
    public function __construct(int $perjanjianKinerjaId)
```

```
    {
```

```
        $this->perjanjianKinerjaId = $perjanjianKinerjaId;
```

```
    }
```

```
    public function handle(PdfGeneratorService $pdfService): void
```

```
    {
```

```
        $pk = PerjanjianKinerja::findOrFail($this->perjanjianKinerjaId);
```

```
        // Generate PDF
```

```
        $pdfService->generate($pk);
```

```
        // Update status
```

```
        $pk->update(['status_dokumen' => 'Generated']);
```

```
        // Send notification
```

```
        send_notification(
```

```
            $pk->pegawai_id,
```

```
            'Dokumen PK Siap',
```

```
            'Dokumen Perjanjian Kinerja Anda sudah siap untuk ditandatangani',
```

```
            'Dokumen',
```

```
            route('perjanjian-kinerja.show', $pk->id)
```

```
        );
```

```
    }
```

```
}
```

Dispatch Job

```
php
```

```
use App\Jobs\ProcessPerjanjianKinerja;

// Dispatch job
ProcessPerjanjianKinerja::dispatch($perjanjianKinerja->id);

// Dispatch with delay
ProcessPerjanjianKinerja::dispatch($perjanjianKinerja->id)
->delay(now()->addMinutes(5));
```

Run Queue Worker

```
bash

# Run queue worker
php artisan queue:work

# Run with specific connection
php artisan queue:work database

# Run specific queue
php artisan queue:work --queue=high,default,low

# Listen for new jobs
php artisan queue:listen
```

KESIMPULAN AKHIR

Tutorial lengkap ini telah mencakup semua aspek pengembangan Sistem TERDEPAN dari A sampai Z:

Yang Telah Dipelajari:

1. **Setup & Konfigurasi**
 - Laravel 11 installation
 - Laravel Modules setup
 - Database configuration
 - Folder structure
2. **7 Modul Aplikasi**
 - Inti (Master Data)
 - Dokumen (File Management)
 - Perjanjian Kinerja (Performance Contract)
 - Penugasan (Task Management)
 - Evaluasi (Assessment)
 - Sistem (Infrastructure)
 - Dashboard (Reporting)
3. **Database Design**
 - 32 tabel lengkap
 - Relationships & Foreign Keys
 - Indexes untuk performance

- Soft deletes & timestamps

4. Backend Development

- Models dengan relationships
- Controllers dengan CRUD
- Services layer
- Repository pattern
- Request validation
- API development

5. Best Practices

- PSR-12 coding standards
- Service layer separation
- Repository pattern
- Helper functions
- Error handling
- Queue & jobs

6. Testing

- Unit testing
- Feature testing
- Database testing
- Migration testing

7. Deployment

- Production configuration
- Server setup
- Security checklist
- Monitoring & logging
- Backup strategy



Next Steps:

1. Implement authentication & authorization
2. Create frontend views dengan Blade/Vue/React
3. Add real-time features dengan WebSockets
4. Implement export to Excel/PDF
5. Add email notifications
6. Create mobile app API
7. Performance optimization
8. Load testing
9. Security audit
10. User training & documentation



Support & Resources:

- Laravel Documentation: <https://laravel.com/docs>

- Laravel Modules: <https://nwidart.com/laravel-modules>
- Stack Overflow: Tag `laravel` dan `laravel-modules`
- GitHub: Submit issues dan pull requests

Selamat! Anda telah menyelesaikan tutorial lengkap Sistem TERDEPAN BAPPEDA Kuningan! 🎉

Happy Coding & Good Luck! 🚀